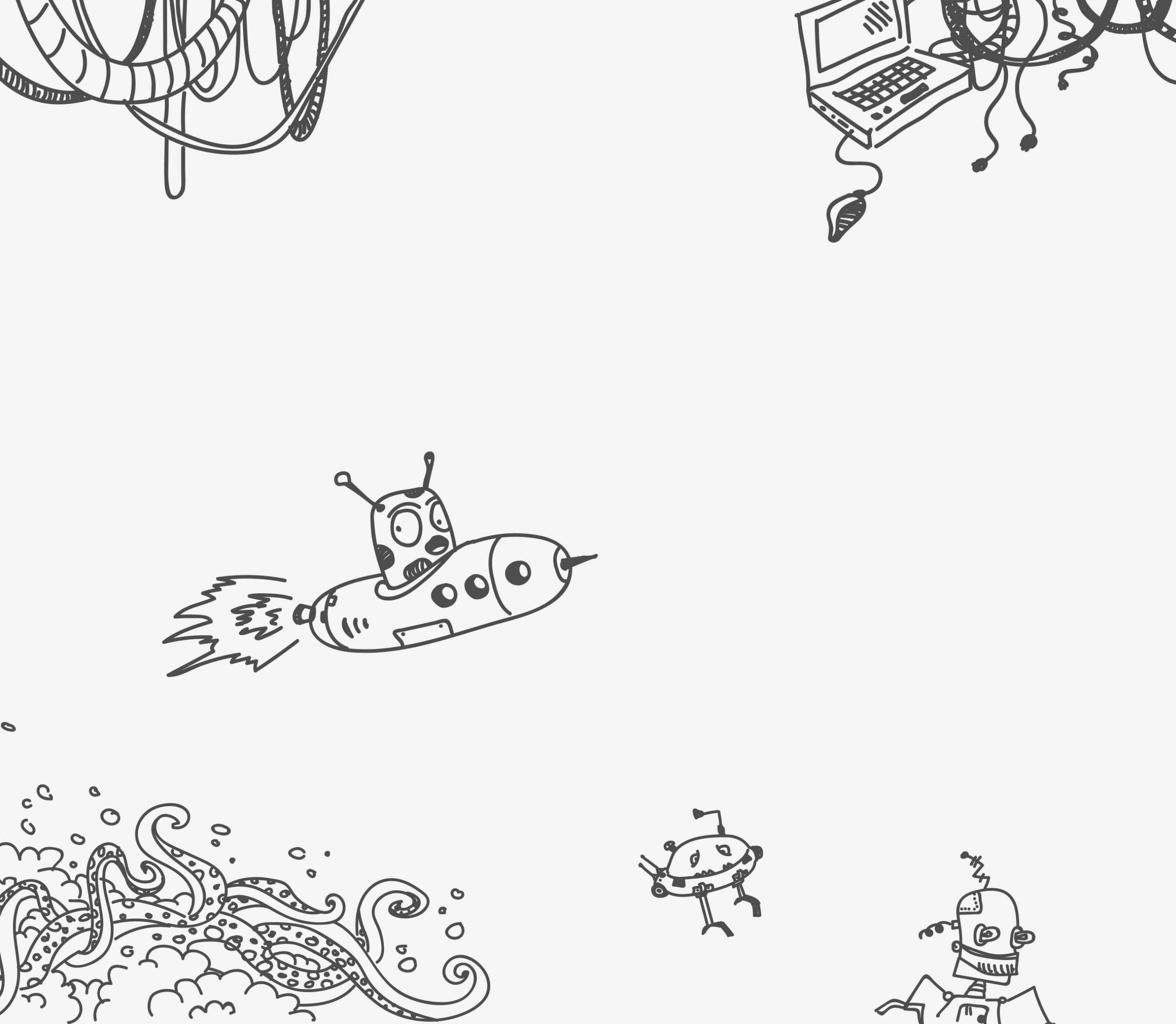


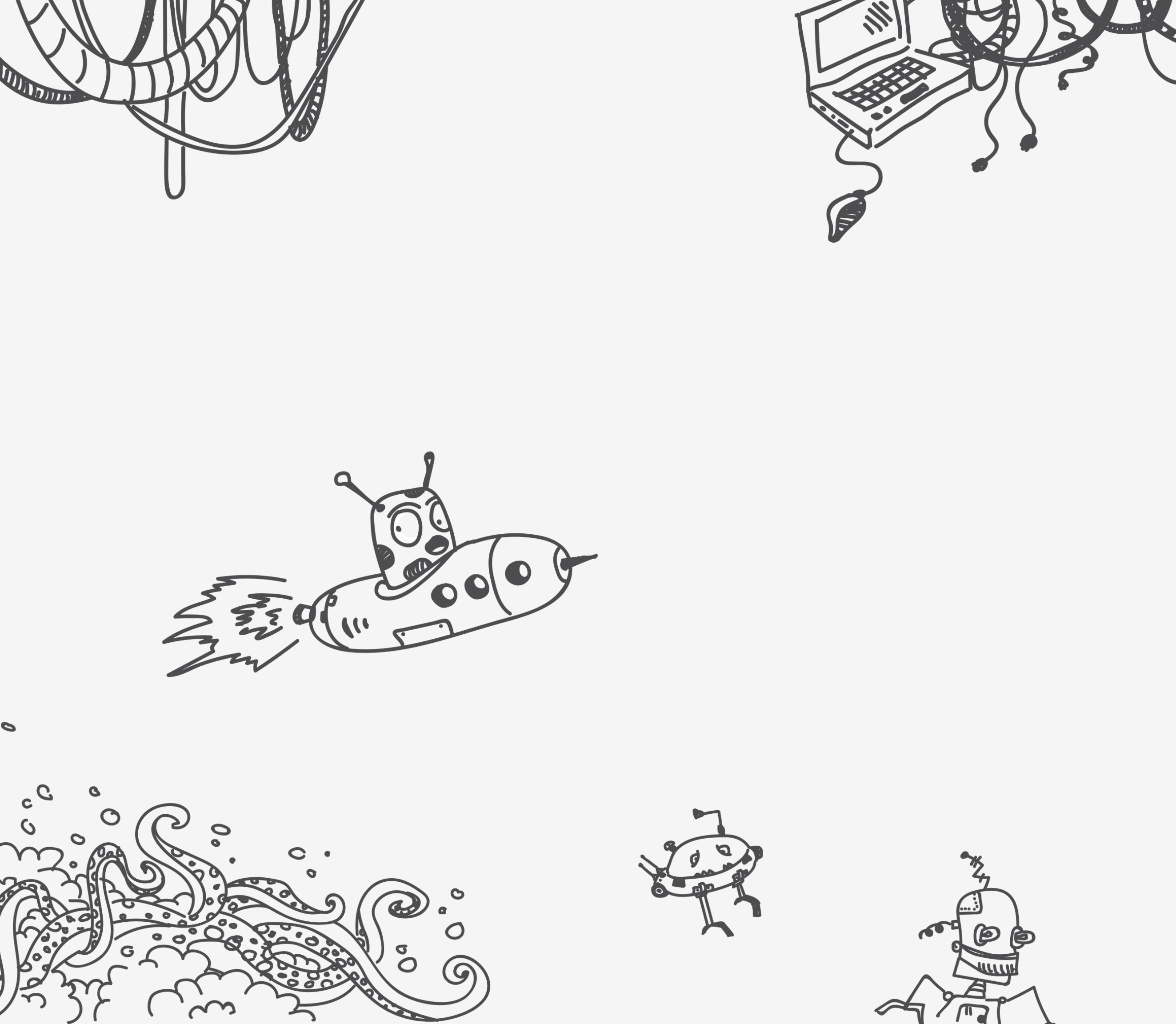
Regularization in a linear regression model - introduction and implementation



Introduction

Introduction

When building machine learning models, we struggle with many challenges to ultimately create the most accurate model possible. One of these is the trade-off between variance and bias. When a model is overfit (has too much variance), it is characterized by poor generalizability in the context of new data. One way to deal with model *overfitting* is **regularization**.



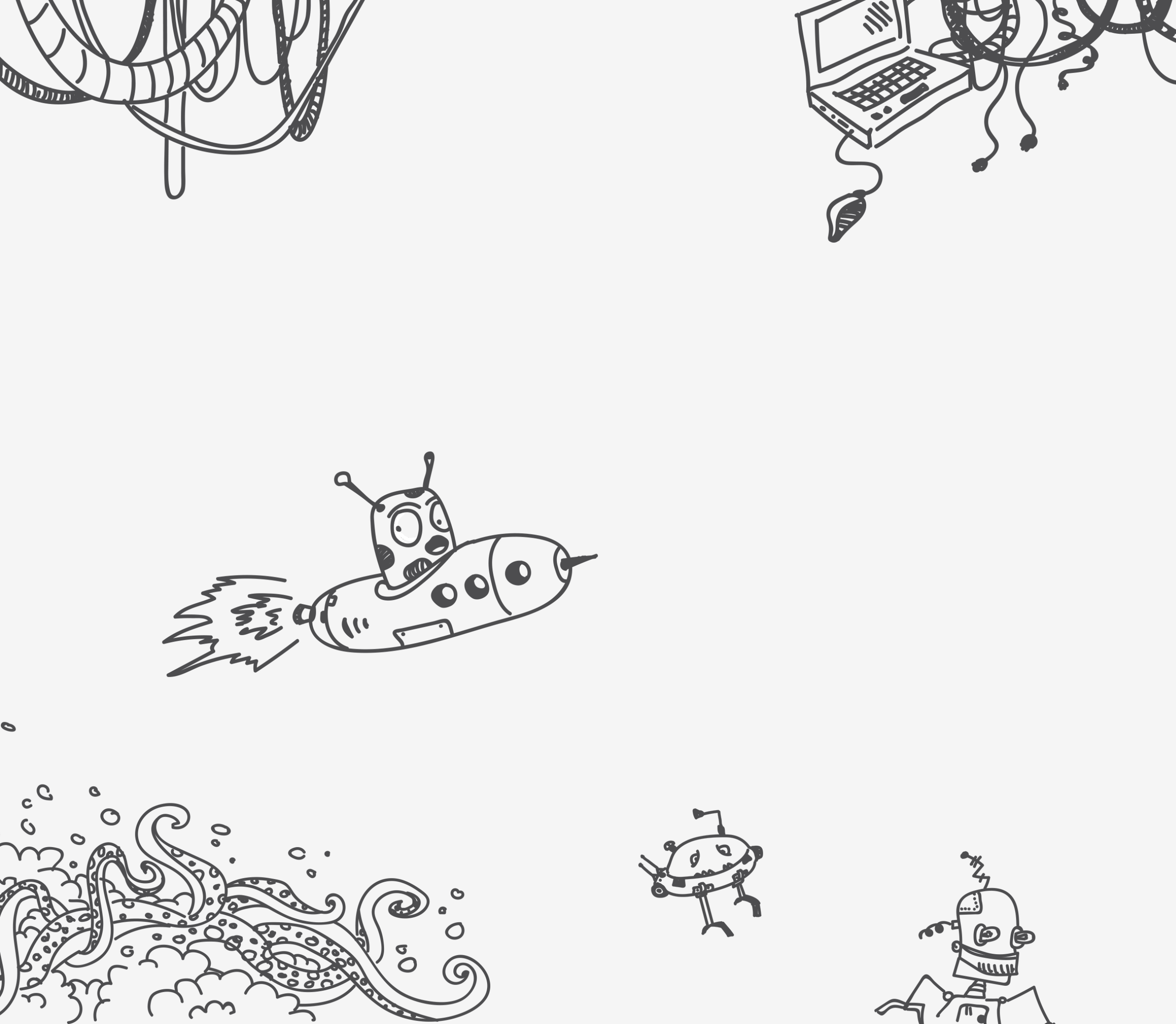
Regularization - what is it?

Regularization - what is it?

Regularization is a technique used in machine learning to prevent the model from overtraining, thereby improving its ability to generalize.

Regularization involves adding a penalty to the cost function in such a way as to reduce the values of the model's parameters (e.g., directional coefficients in a linear regression) without adversely affecting its ability to fit the training data.

We have various techniques at our disposal, such as Lasso, Ridge, Elastic Net and others. We will first focus on the first two regularization methods also known as **L1** (Lasso) and **L2** (Ridge).



Lasso regularization (L1)

Lasso regularization (L1)

Lasso regularization is a way of helping the model to select only those things that are important and omit the unnecessary ones. It can be compared to packing for a trip to the mountains - you want to take only the most important things, so that the backpack you travel with for many kilometers is not too heavy. Similarly, a model using Lasso regularization selects only the most important features in the data and skips the unnecessary ones. This allows the model to work better and faster.

Lasso regularization involves adding the sum of the absolute values of the parameter weights multiplied by the **lambda** regularization constant to the cost function. The very general form of the cost function for regularization is as follows:

$$\text{cost}_{function} = \text{loss} + \text{regularization}$$

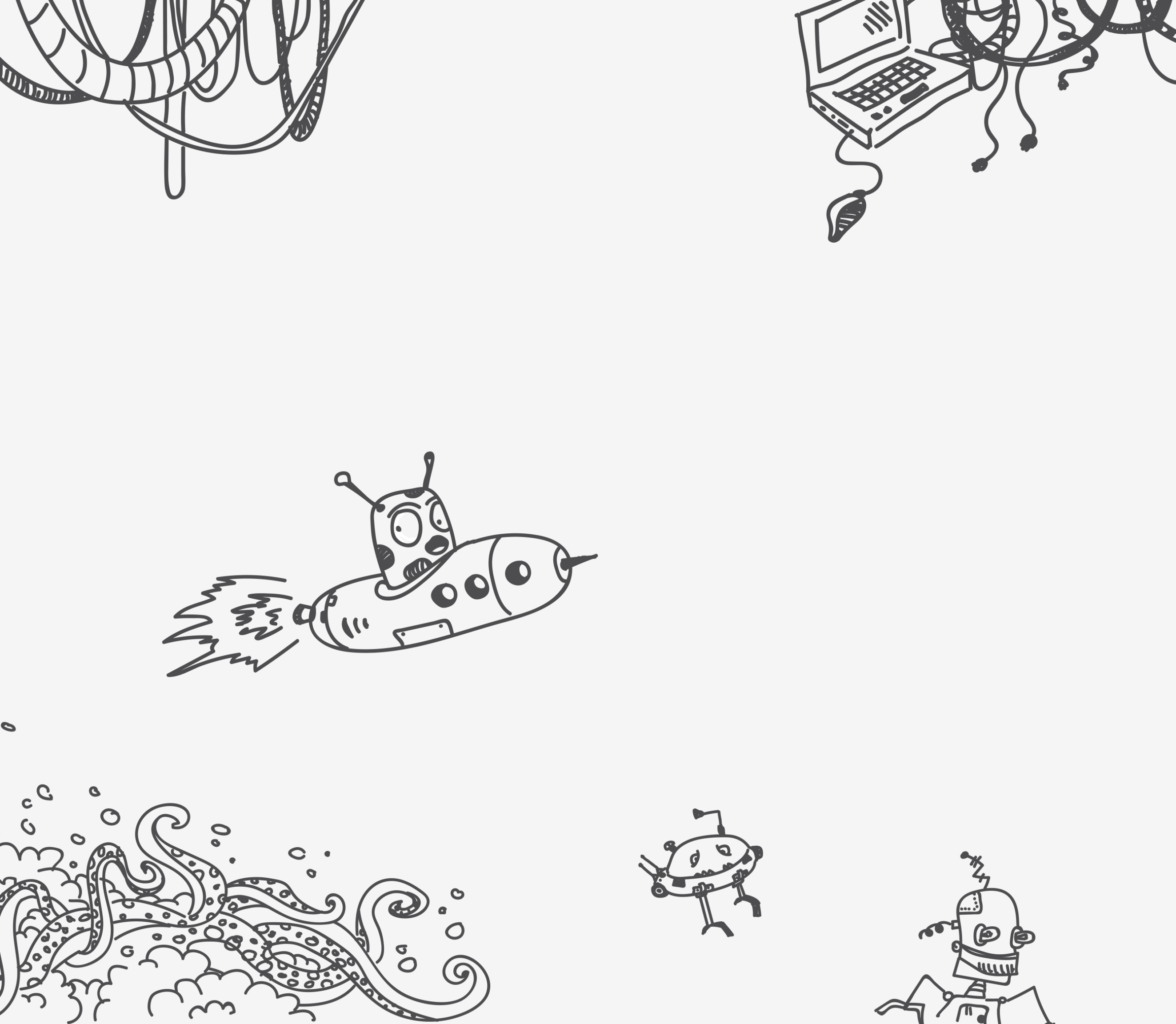
.

The above formula has an expansion, which looks as follows:

$$J(\beta) = MSE + \lambda * \sum (\beta_i)$$

where:

- **MSE** is the mean squared error, which is the cost function of a linear regression without regularization,
- β is the vector of weights (coefficients $a_0, a_1, a_2 \dots$ etc.) of the linear regression model,



Lasso regularization (L1) in Python

Lasso regularization (L1) in Python

Let's see how we can easily create a linear regression model using Lasso regularization. All we need to do is import the **Lasso** class from the `linear_model` module, and then initialize an object of this class as our model. With regularization, we encounter the concept of a hyperparameter, which is *alpha*. Alpha is a regularization constant that determines how strongly we will penalize the coefficients - the higher the value, the greater the penalty.

```
from sklearn.linear_model import Lasso

# Create model based on Lasso regularization
lasso_reg = Lasso(alpha=0.1)
```

Next, we follow the already standard procedure - using the `.fit()` method, we train our model and then make a prediction.

```
# Fit the model on training data
lasso_reg.fit(X_train, y_train)
```

Lasso regularization (L1) in Python

Let's compare the model from the Scikit-learn linear regression implementation example with the model using regularization:

```
# Predict values based on test data  
y_pred_lasso = lasso_reg.predict(X_test)
```

The mean squared error for the basic model was 0.53 (in hundreds of thousands of dollars), and for the model with Lasso regularization it was 0.597. This means that the model with regularization is slightly worse. However, let's try to reduce the regularization strength by one order of magnitude:

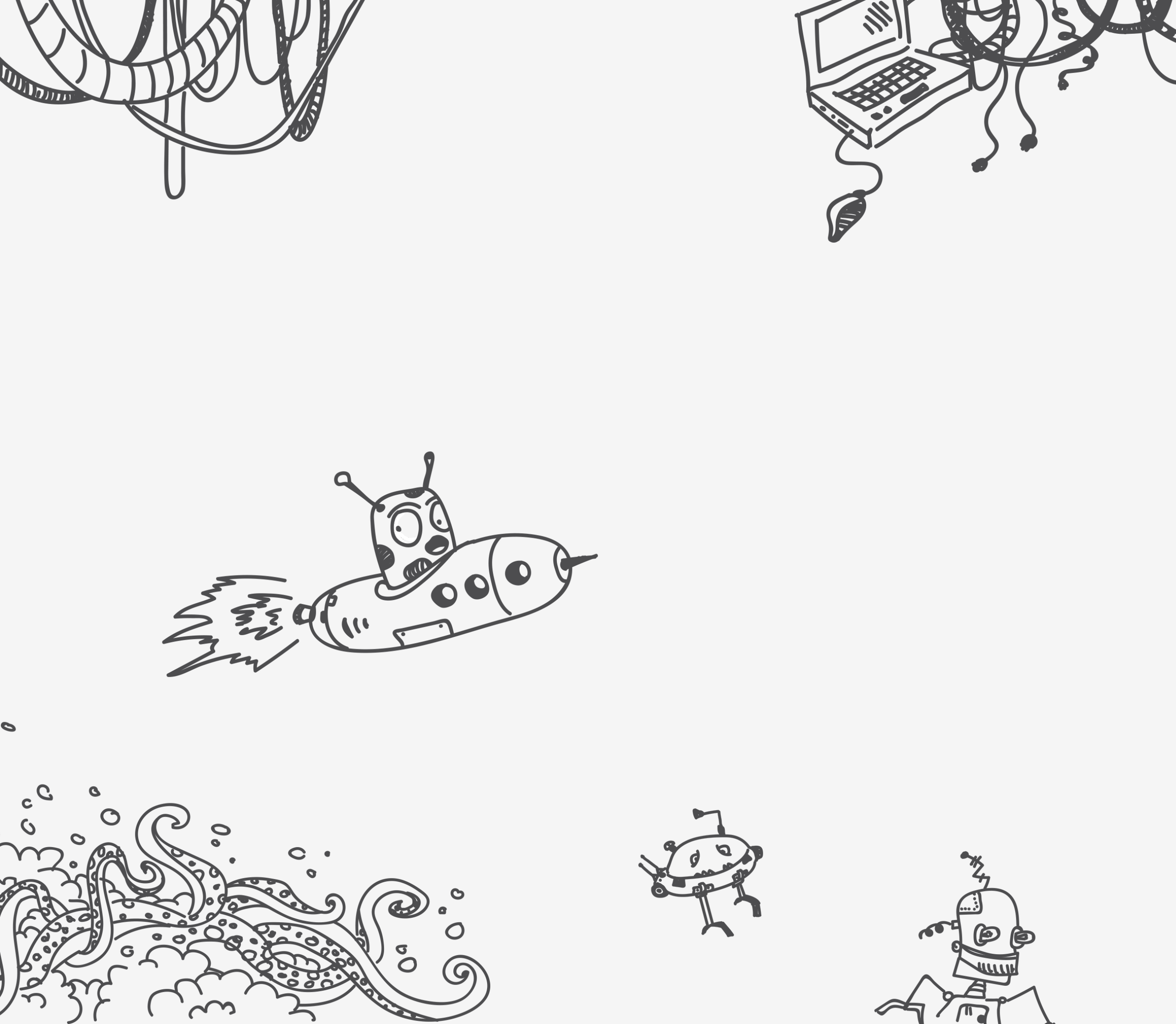
```
# Initialize model with different hyperparameter value  
lasso_reg = Lasso(alpha=0.01)
```

Lasso regularization (L1) in Python

After changing the regularization strength, the MSE for our model was 0.525. Of course, this is still a big error (\$525,000!), but we see an improvement over the base model. Let's see if another reduction in the strength of regularization will improve the result.

```
# Initialize model with different hyperparameter value  
lasso_reg = Lasso(alpha=0.001)
```

After calculating the MSE, we see that the error value has increased. Reducing the strength of regularization further does not make sense, as it has no positive effect.



Ridge regularization (L2)

Ridge regularization (L2)

The second popular method of regularization is **Ridge** regularization, also known as L2. Ridge regularization is such a tool that helps the model focus on things that are similar, but at the same time reduces the impact of differences between them. It's a bit like going on a hike in the mountains with a backpack into which you pack all your prepared items. However, you later come to the conclusion that some things (e.g., food) you need to take more than others (e.g., changing clothes).

Ridge regularization (L2)

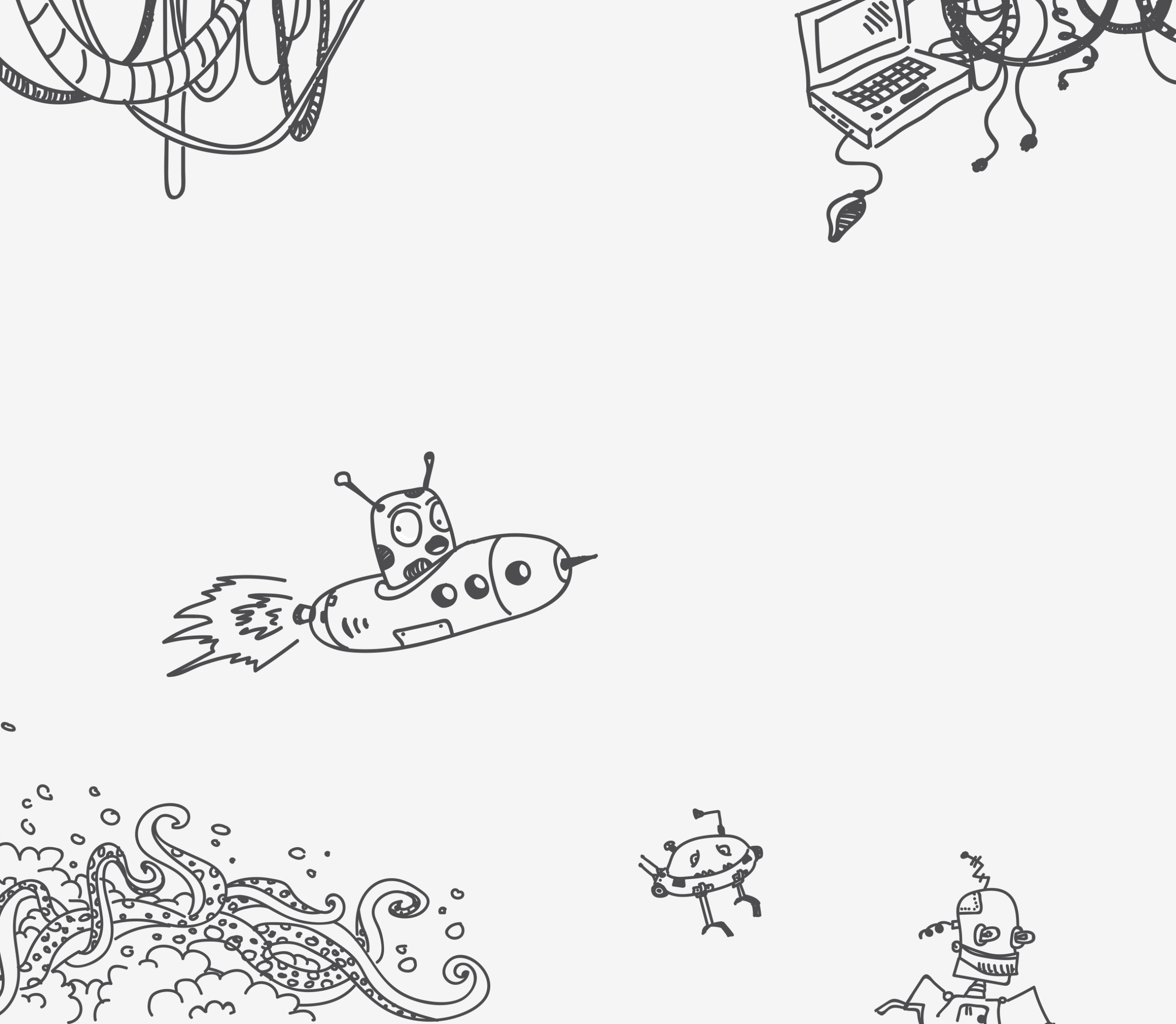
Ridge regularized model does not remove variables from the model, but seeks to equalize the values of the weights to somewhat reduce their influence on the model.

$$J(\beta) = MSE + \lambda * \sum (\beta_i)^2$$

where:

- ***MSE*** is the mean squared error, which is the cost function of a linear regression without regularization,
- ***β*** is the vector of weights (coefficients $a_0, a_1, a_2 \dots$ etc.) of the linear regression model,
- ***λ*** is the regularization constant that determines its strength,
- **sum(*β_i*)²** determines the sum of the squares of the values of the model parameters' weights.

To the cost function, we add the sum of squares of the model weights multiplied by the alpha, which is the strength of the regularization. One change from the **Lasso** regression can be seen here: we add the sum of squares of the weights (coefficients), rather than the sum of the absolute values of the weights (coefficients). Due to the minimization of the sum of squares of the values of the weights, the model is forced to choose less complex solutions, which prevents overtraining of the model.



Ridge regularization (L2) in Python

Ridge regularization (L2) in Python

The implementation of the model with L2 regularization is actually analogous to the implementation of L1 regularization. Only the imported class differs (Lasso -> Ridge). The Ridge class also takes an alpha argument, which determines the strength of the regularization. Let's arbitrarily set alpha to 10. After making the prediction, let's check what kind of mean square error we got this time:

```
from sklearn.linear_model import Ridge

# Create model based on Ridge regularization
ridge_red = Ridge(alpha=10)

# Fit the model on training data
ridge_red.fit(X_train, y_train)

# Predict values based on test data
y_pred_ridge = ridge_red.predict(X_test)

# Calculate MSE
mean_squared_error(y_test, y_pred_ridge)
```

Ridge regularization (L2) in Python

By increasing the alpha parameter, the mean-square error decreased. To find the optimal value of the alpha parameter, we can define a list of values and use *for* loops. Another way could be crossvalidation and grid or random search.

We will talk more about these techniques later in the course.

```
from sklearn.linear_model import Ridge

alphas = [0.001, 0.01, 0.1, 1, 10, 100, 1000]

for alpha in alphas:
    # Create model based on Ridge regularization
    ridge_red = Ridge(alpha=alpha)

    # Fit the model on training data
    ridge_red.fit(X_train, y_train)

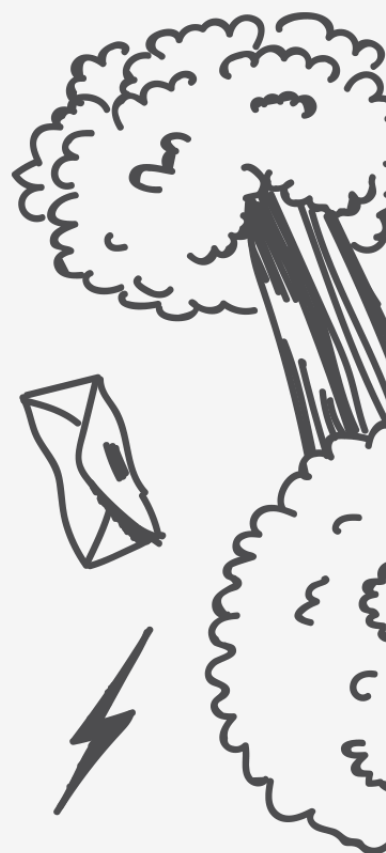
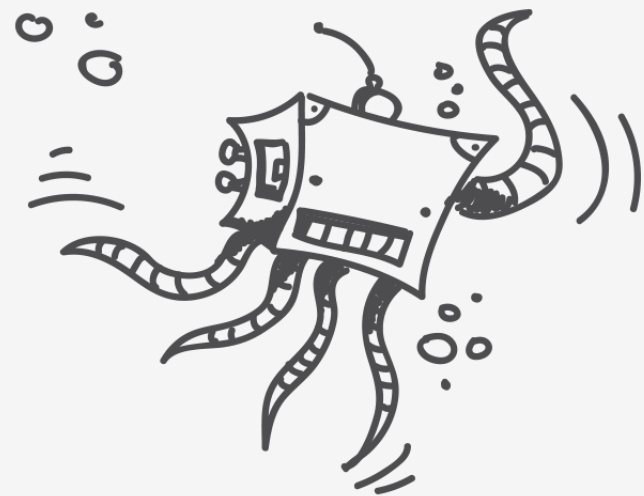
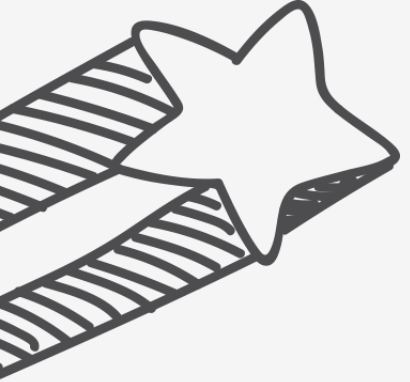
    # Predict values based on test data
    y_train_ridge = ridge_red.predict(X_train)
    y_pred_ridge = ridge_red.predict(X_test)

    # Calculate MSE
```

Summary

During the presentation, we learned about the concept of regularization. We know what types of regularization there are, and how the discussed **Lasso** and **Ridge** regularizations differ.

Lasso (L1) regularization is a good choice when the data we are working with has too many variables. On the other hand, **Ridge** regularization is the right choice when the variables are few, but we have a clearly overtrained model.



Practice time